DAY 26 | HANDS-ON ACTIVITIES

LLM Orchestration, Tool Calling and LangGraph: Problem Statements

Eight problems. Every notebook cell runs offline with a rule-based stand-in model; add an OPENAI_API_KEY to see real tool choices. Offline results show how the code works, not real model quality.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Summary Chain for Three Audiences

Level: Warm-up | **Concepts:** ChatPromptTemplate, LCEL, batch

Build a chain that summarises a support incident for a given audience (CIO, engineer, end user) in 3 bullet points, and run it on one text for all three audiences with batch().

Sample run / expected result

```
CIO: cost and risk focus
Engineer: technical detail
End user: what changes for me
```

Hints

- prompt | model | StrOutputParser()

Problem 2: Write Tool Schemas

Level: Easy | **Concepts:** JSON schema, descriptions

Write schemas for get_order_status(order_id), calculate_refund(amount, days_late) and create_ticket(summary, priority). Include descriptions with example values.

Sample run / expected result

```
{"name": "calculate_refund", "parameters": {"properties": {"amount": {"type": "number"}, ...}
```

Hints

- Use enum for priority: low, medium, high.

Problem 3: Validate Arguments

Level: Easy | **Concepts:** Pydantic, allowlist

Write run_tool(name, args_json) that rejects unknown tools and invalid arguments (bad order id format, negative amount) with clear error messages returned to the model.

Sample run / expected result

```
run_tool("delete_db", "{}") -> error: unknown tool
run_tool("calculate_refund", '{"amount": -5}') -> error: amount must be > 0
```

Hints

- Return errors as text so the model can recover.

Problem 4: Agent with Middleware NEW (v4)

Level: Medium | **Concepts:** create_agent, middleware

Build a create_agent assistant with get_order_status and calculate_refund. Add a @before_model logger, a @wrap_tool_call audit log and a ModelCallLimitMiddleware. Run it on two questions.

Sample run / expected result

```
[log] model call 1 (1 message)
[audit] get_order_status {'order_id': 'ORD-1042'} -> ok
[log] model call 2 (3 messages)
FINAL: refund of 720
```

Hints

- The handler in wrap_tool_call runs the tool; log before and after it.

Problem 5: Tool Token Budget NEW (v4)

Level: Medium | **Concepts:** tool token cost, tool design

Count the tokens of your tool list. Compare 3 lean tools with 12 verbose tools, and estimate the cost per 1,000 conversations of 4 model calls each. Use the notebook's placeholder prices.

Sample run / expected result

```
lean (2 to 3 tools): about 150 tokens per call
verbose (12 tools) : about 2,450 tokens per call
extra per 1,000 conversations (4 calls each): about 9.2 (placeholder units)
```

Hints

- Tool definitions are sent on every model call.
- Label prices as placeholders.

Problem 6: Same Assistant, Three Ways NEW (v4)

Level: Medium | **Concepts:** direct API, LangChain, Agents SDK

Implement the order-status question with the direct API loop, create_agent and the OpenAI Agents SDK. Fill a table: lines of code, where validation lives, how you would test it, how you see a trace.

Sample run / expected result

```
| option     | lines | validation    | trace          |
| direct API | 45    | run_tool      | your own logs  |
| LangChain  | 20    | tool + middle | LangSmith/logs |
| Agents SDK | 15    | function_tool | built-in trace |
```

Hints

- The Agents SDK run needs a key; describe the offline result from the notebook.

Problem 7: Support Graph with Retry

Level: Challenging | **Concepts:** LangGraph, conditional edges

Build a graph: classify -> (order | policy | other) -> answer -> check. If check fails, retry once with a rewritten question, then hand off to a human. Print the Mermaid diagram.

Sample run / expected result

```
classify: order -> lookup -> answer -> check: pass
classify: policy -> retrieve -> answer -> check: fail -> retry -> pass
```

Hints

- Keep attempts in state to stop loops.
- app.get_graph().draw_mermaid()

Problem 8: Durable Approval NEW (v4)

Level: Stretch | **Concepts:** checkpoints, interrupt, Command(resume)

Add an approve_refund node that calls interrupt(). Run until it pauses, list the checkpoints, then resume once with 'no' and once (on a new thread) with 'yes'. Log every decision.

Sample run / expected result

```
paused at: approve_refund {'amount': 720.0}
checkpoints: 5
thread A resume 'no'  -> refund rejected
thread B resume 'yes' -> refund issued
```

Hints

- Use InMemorySaver and a thread_id per conversation.
- Default to no.